

ggmlR: Vulkan GPU-бэкенд для R

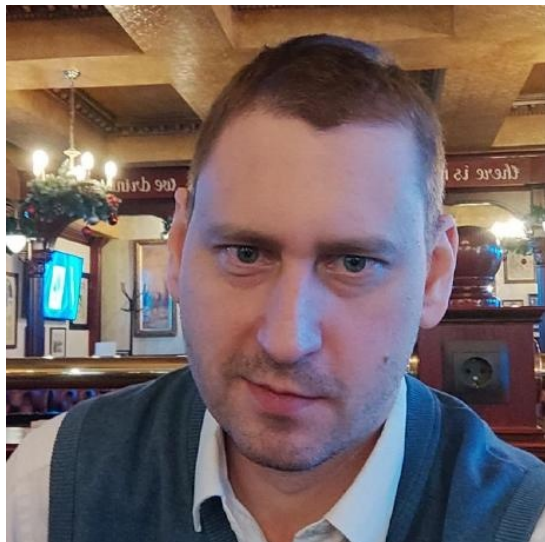
О СПИКЕРЕ

Юрий Барамыков

Старший руководитель проекта
Иннотех

Управление проектами, разработка и внедрение ПО в Банках.
Хобби — разработка пакетов на R

ОПЫТ	15 лет в ИТ
СТЕК	R, C/C++, GLSL
КОНТАКТ	lbsbmsu@mail.ru



Содержание

- Описание проблемы
- С чего всё началось
- GGML Georgi Gerganov
- ggmlR описание по слоям
- ONNX: запуск моделей на Kaggle
- llamaR - Скорость работы LLM
- sd2R — flux1-dev-Q4_K_S
- Подключение Vulkan-бэкенда ggmlR
- Выводы и планы на будущее

Слабое место R - GPU-экосистема

Проблема R

GPU-обучение (**TensorFlow, PyTorch**) исторически развивалось в Python-стеке на **NVIDIA/CUDA**.

В R глубокое обучение на GPU возможно, но обычно требует обходных путей через **reticulate, Python и CUDA** и почти не покрывает пользователей с GPU AMD или Intel.

ggmlR - это попытка вернуть глубокое обучение в R на современном оборудовании, не завязанном на стек **CUDA**.

С чего всё началось

Работал с R — но чего-то не хватало.

Не нашел подходящие инструменты для GPU-ускорения.

Попытался запустить **Stable Diffusion** на Python + CUDA → несколько часов, боль, результат разочаровал.

Что хотелось от продукта

3 требования:

Легко установить — ставится 1 пакет R

Высокая скорость — ускорение GPU работает быстрее чем CPU

Поддержка разных GPU — например, можно запускать на Kaggle

Поддержка GPU AMD в R

Пакеты на базе OpenCL (Максимальная совместимость)

OpenCL - предоставляет низкоуровневый доступ для прямого запуска собственного кода (ядер) на видеокарте.

lightgbm - реализует высокопроизводительный градиентный бустинг на деревьях решений с поддержкой ускорения через OpenCL.

glmblayes - предназначен для ускорения расчетов в байесовских обобщенных линейных моделях.

Пакеты на базе AMD ROCm / HIP (Максимальная производительность)

torch - обеспечивает полноценное обучение нейросетей и выполнение любых тензорных вычислений напрямую через C++ библиотеку LibTorch.

xgboost - представляет собой популярную библиотеку для задач классификации и регрессии с возможностью переноса вычислений на GPU

Топ пакеты в R написаны на C/C++

Почти у всех топ-20 популярных пакетов под капотом — **C/C++**.
Значит, нужно найти хороший **C-проект** и перенести его в R
для использования **GPU**

Топ-5

Пакет	Язык (Engine)	Основная функция
Rcpp	C++	Интеграция C++ кода в среду R
dplyr	C++	Быстрая манипуляция данными
data.table	C	Работа с таблицами
stringi	C++ (ICU)	Обработка текста
jsonlite	C (yajl)	Парсинг и генерация JSON

GGML Georgi Gerganov

Что такое GGML, почему этот проект — **идеальный кандидат**.

Уже поддерживает GPU, активно развивается, MIT-лицензия.

Кроссплатформенность: библиотека оптимизирована для эффективной работы на обычном потребительском оборудовании.

Минимализм: код написан на чистом **C** без тяжелых внешних зависимостей.

Выбор GPU-технологии

GGML поддерживал разные форматы, нужно было выбрать

Критерии выбора

Разработчик	NVIDIA CUDA	AMD ROCm	Khronos Vulkan (Compute)
Тип лицензии	Проприетарная	Открытая (Open Source)	Открытый стандарт (Open Source)
Поддерживаемое железо	Только GPU NVIDIA	Только GPU AMD	Любые GPU (NVIDIA, AMD, Intel, Apple, мобильные чипы ARM/Qualcomm)
Портируемость кода	Нулевая (привязана к экосистеме NVIDIA)	Высокая на AMD (есть инструмент HIP для конвертации кода CUDA)	Абсолютная (один код работает на любом устройстве)

Выбор был сделан в пользу Vulkan

Как опубликовать пакет в CRAN

Все пакеты проходят строгую ручную проверку людьми.

Сдать пакет в **CRAN** сложнее чем в **PyPI** или **npm**, но качество программ здесь выше.

Шаги по созданию пакета

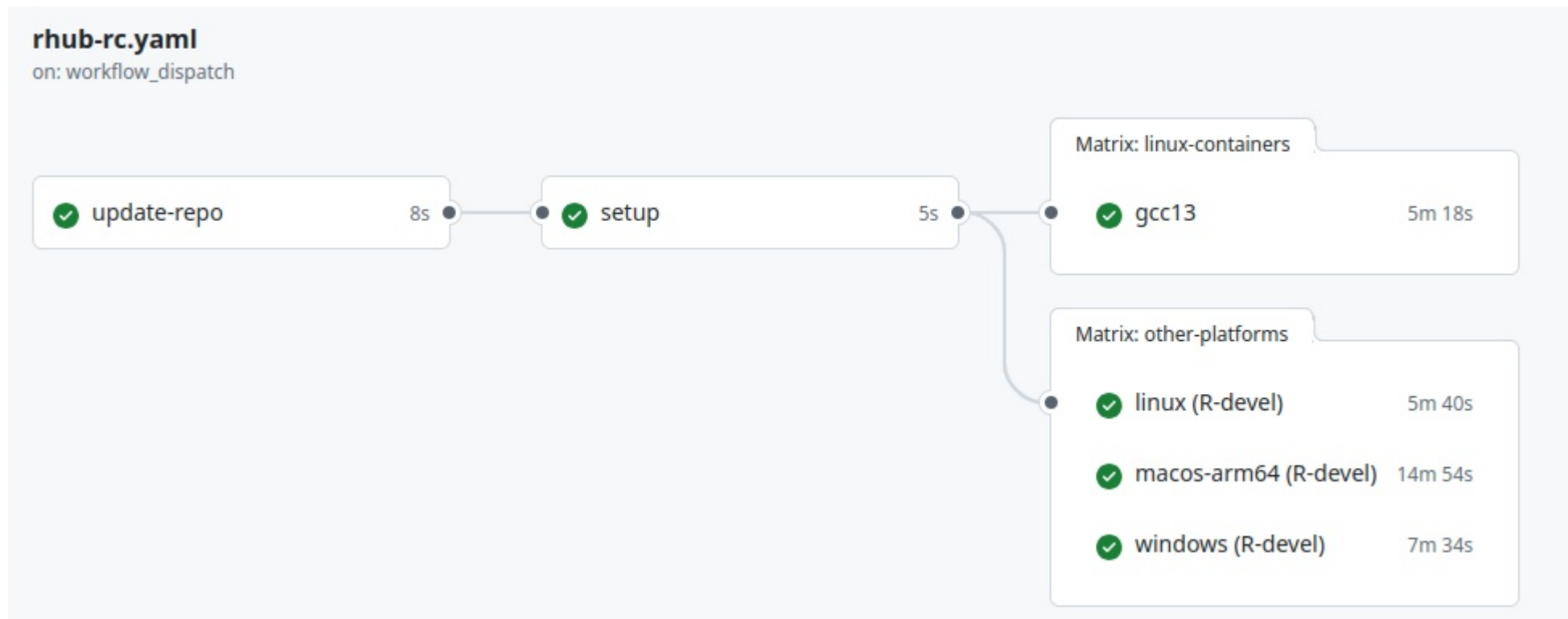
- 1) Делаем пакет (код, документация, тесты, примеры)
- 2) Все юнит-тесты проходят
- 3) Автоматический тест (R CMD check)
- 4) R-hub — сборка и R CMD check на разных платформах
- 5) Проверка на Kaggle
- 6) Отправка в CRAN (авто-тесты)
- 7) Контрольная проверка от члена команды CRAN
- 8) Контрольная проверка на разных средах и сборках R

Есть ограничения

На серверах CRAN нет **GPU**

Есть ограничение по размеру пакета в **5 mb**.

Пример работы пайпа тестирования



ggmlR описание по слоям

R API
`load_model()`, `generate()`

R API — пользователь работает с привычными R-функциями: `load_model()`, `generate()`, `embed()`. Никакого Python, никакого conda

Rcpp / C++ Bridge
Zero-copy · no overhead

Rcpp / C++ bridge — R вызывает C++ напрямую через Rcpp. Zero-copy передача данных — нет лишних аллокаций при передаче весов

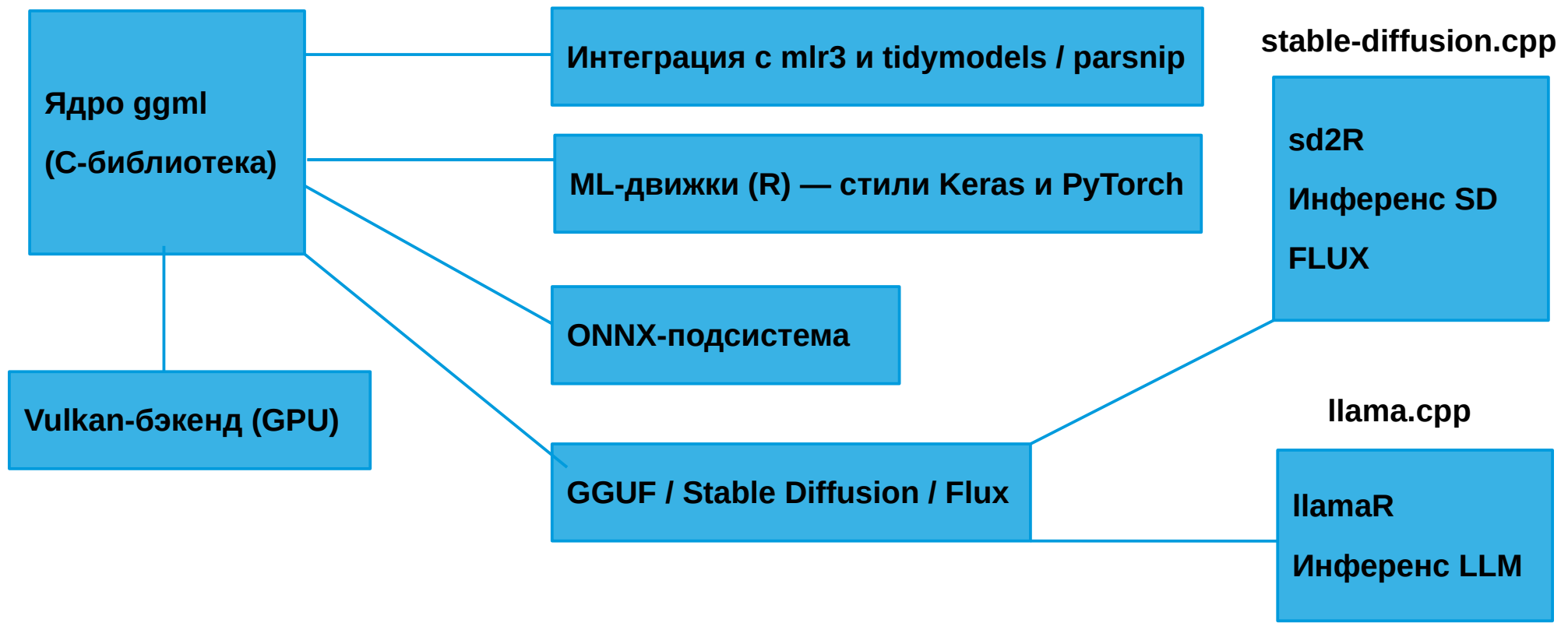
GGML Core
Квантизация Q4/Q8

GGML core — библиотека тензорных вычислений от автора llama.cpp. Поддерживает квантизацию Q4/Q8 — модель 7B умещается в 4 ГБ VRAM

Vulkan Backend
Compute shaders

Vulkan backend — GPU-ускорение через Vulkan compute shaders. Работает на AMD и NVIDIA без CUDA — **открытый** стандарт для GPU вычислений

ggmlR архитектура



ONNX: запуск моделей на Kaggle

Model	CPU(ms)	GPU(ms)	Speedup	CPU FPS	GPU FPS
-----	-----	-----	-----	-----	-----
Inception V3	799.0	73.3	10.90x	1.3	13.6
MNIST	0.7	1.0	0.67x	1500.0	1000.0
SqueezeNet 1.0	100.0	10.0	10.00x	10.0	100.0
SuperResolution	523.0	34.0	15.38x	1.9	29.4
EmotionFerPlus	131.0	17.3	7.56x	7.6	57.7
Inception V3 Op18	788.3	73.0	10.80x	1.3	13.7
BAT-ResNeXt26ts	542.3	52.0	10.43x	1.8	19.2
BERT (Opset17)	1356.0	40.7	33.34x	0.7	24.6
GPT-NeoX	11.0	2.7	4.13x	90.9	375.0



ONNX

50+ операторов

Открытый формат для
нейросетевых графов.

Экспортируется из PyTorch,
TensorFlow, scikit-learn.

LlamaR - Скорость работы LLM

Qwen3-14B-Q8_0.gguf
Llama-bench AMD RX 9070
GPU: **24.47** tokens/sec

ggmlR AMD RX 9070
GPU: **22.4** tokens/sec

Qwen3-14B-Q8_0.gguf
ggmlR AMD RX 9070
=== Results ===
CPU: 2.0 tokens/sec
GPU: 22.4 tokens/sec
Speedup: 11.39x

llama3-8b-instruct-Q8_0.gguf

Запуск LLM работает на GPU и CPU
Vulkan установлен на Kaggle
GPU **Nvidia Tesla P100**

ggmlR AMD RX 9070
=== Results ===
CPU: 4.3 tokens/sec
GPU: 70.0 tokens/sec
Speedup: 16.14x

Nvidia Tesla P100
=== Results ===
CPU: 2.3 tokens/sec
GPU: 32.7 tokens/sec
Speedup: 14.26x



ggmlR - llamaR - Opencode

Локальный запуск Opencode через llamaR

- Составьте пример кода/логики.
- Добавьте/обновите настройки.
- Настройте интеграцию с другими инструментами.
- Добавить тесты.

/INST Приведите пример кода или условия в формате:

- [описание задачи]

Или просто напишите условие.

Советую подробно описать шаги и что именно должно быть обновлено в конфигурации.

■ Build · Ministral-3B (llamaR) · 1.8s

■ Build · Ministral-3B (llamaR) llamaR (local)

tab agents ctrl+p commands

```
yur@yur1-0400-kvms-1111:~$ Rscript /mnt/Data2/DS_projects/llamaR/inst/example
s/serve_openai.R /mnt/Data2/DS_projects/llm_models/Ministral-3-3B-Instruct-2512-
Q4_K_M.gguf 11434 16384
Загрузка требуемого пакета: ggmlR
WARNING: radv is not a conformant Vulkan implementation, testing use only.
llamaR OpenAI server: http://127.0.0.1:11434 (model 'Ministral-3-3B-Instruct-25
12-Q4_K_M')
drogonR listening on http://0.0.0.0:11434 (threads=1, workers=1, routes=2, cpp_r
outes=0, cpp_stream_routes=0, static=0)
20260527 03:27:28.890438 UTC 626937 DEBUG [makeHeaderString] send stream with tr
ansfer-encoding chunked - HttpResponseImpl.cc:756
20260527 03:27:29.616998 UTC 626937 DEBUG [makeHeaderString] send stream with tr
ansfer-encoding chunked - HttpResponseImpl.cc:756
20260527 03:27:45.695933 UTC 626937 DEBUG [makeHeaderString] send stream with tr
ansfer-encoding chunked - HttpResponseImpl.cc:756
20260527 03:28:00.357061 UTC 626937 DEBUG [makeHeaderString] send stream with tr
ansfer-encoding chunked - HttpResponseImpl.cc:756
```

ggmlR - llamaR - chat

```

...: /mnt/Data2/DS_projects/llamaR
** byte-compile and prepare package for lazy loading
** help
*** installing help indices
** building package indices
** installing vignettes
** testing if installed package can be loaded from temporary location
** checking absolute paths in shared objects and dynamic libraries
** testing if installed package can be loaded from final location
** testing if installed package keeps a record of temporary installation path
* DONE (llamaR)
...E:/mnt/Data2/DS_projects/llamaR$ Rscript /mnt/Data2/DS_projects/llamaR/inst/examples/chat.R /mnt/Data2/DS_projects/llm_models/Ministral-3-3B-Instruct-2512-Q4_K_M.gguf --system "Ты краткий ассистент, отвечай по-русски."
Загрузка требуемого пакета: ggmlR
Loading /mnt/Data2/DS_projects/llm_models/Ministral-3-3B-Instruct-2512-Q4_K_M.gguf and starting a server on port 11434 (timeout 180s) ...
Ready. Type a message and press Enter (blank line or Ctrl-D quits).

> Привет! как дела?
Привет! Все хорошо, спасибо. А у тебя всё хорошо? 😊

> 
```

**CLI-чат на
локальной
модели через
llamaR**

sd2R - flux1-dev-Q4_K_S

--- 1. Flux 768x768 -> direct ---

Generated 1 image(s): 768x768 in 72.5s

sd2R Profile

text_encode	6.45s (8.9%)
clip	2.66s (3.7%)
t5	3.78s (5.2%)
sampling	63.21s (87.2%)
vae_decode	9.27s (12.8%)
TOTAL	72.48s

--- 3. Flux 2048x1024 -> auto highres fix ---

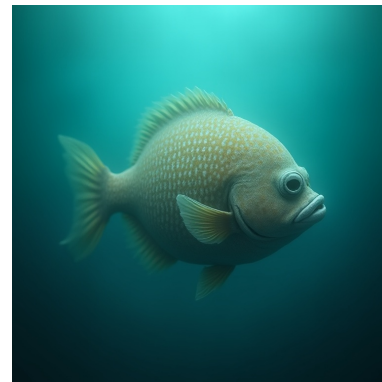
[highres_fix] Step 1: base 1024x512

[highres_fix] Step 2: bilinear upscale to 2048x1024

[highres_fix] Step 3: tiled img2img (strength=0.40, steps=10)

--- Summary ---

text_encode	7.26s (2.9%)
vae_encode	6.47s (2.5%)
sampling	231.26s (91.1%)
vae_decode	22.66s (8.9%)
TOTAL	253.95s

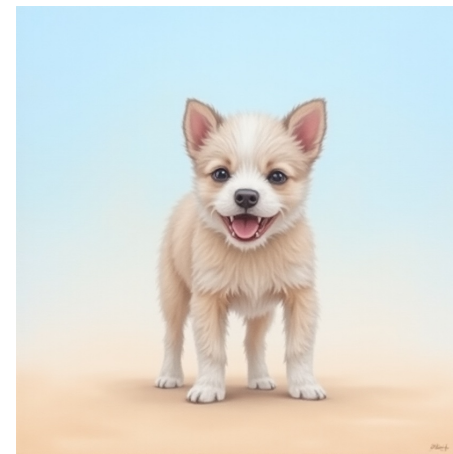


Flux Multi-GPU

Flux Image Generation Benchmark

flux1-dev-Q4_K_S on AMD 9070 (10 steps)

- | | |
|---------------------------|--------|
| 1. 768x768 direct: | 14.13s |
| 2. 1024x1024 tiled VAE: | 25.39s |
| 3. 2048x1024 highres fix: | 52.44s |
| 4. img2img 768x768: | 9.07s |
| 5. 1024x1024 direct: | 25.68s |



sd2R Image Generator

Есть Shiny app для запуска генерации из интерфейса

sd2R Image Generator

GPU

[0] AMD Radeon RX 9070 (RADV GFX1201)
(16.4 / 17.2 GB)

Model

Architecture

flux

Models folder

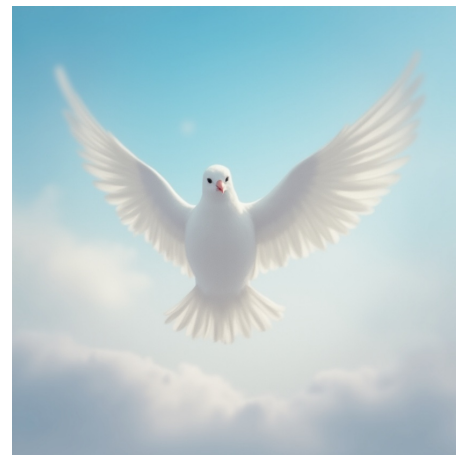
/mnt/Data2/DS_projects/s Scan

Model

flux1-dev-Q4_K_S.gguf

Loading tensors 842/1588 (25s)... model.cpp:1663 - unknown tensor 'single_blocks.5.linear2.bias | f32 | 1 [3072, 1, 1, 1, 1]' in model file

Generated image will appear here



ggmlR - Titanic test

Вариант	Accuracy	F1
1_seq_shallow_sgd	0.7514	0.6271
2_seq_deep_dropout	0.8362	0.7603
3_seq_batchnorm	0.8249	0.7395
4_ag_early_stop	0.7232	0.5950
5_ag_adam_cosine	0.7458	0.6400
6_functional_batchnorm	0.8362	0.7717
7_functional_2inputs	0.8249	0.7395
8_ag_sgd_momentum	0.7119	0.5920
9_ag_adam_cosine_clip	0.6780	0.4956
10_ag_raw_dp_train	0.6667	0.4870



Примеры использования

Данный список примеров показывает усложнение использования нейросетей в **ggmlR** от простых однослойных моделей до сложных функциональных графов с обработкой нескольких типов данных одновременно.

Подключение Vulkan-бэкенда ggmlR



Способ интеграции	Сложность	Скорость работы	Когда выбирать
1. Статическая C++ линковка	● Высокая	⚡ Максимальная	Нужен полный контроль над памятью и C++ логикой
2. Высокоуровневое R-API	● Низкая	● Средняя	Логика полностью пишется на чистом R
3. Интеграция в mlr3	● Средняя	⚡ Высокая	Нужны готовые пайплайны и бенчмарки ML

Три проблемы

Санкции на «не-NVIDIA» железо

Классический Deep Learning (PyTorch/CUDA) жестко привязан к видеокартам NVIDIA.

Родные инструменты для Data Science

Аналитики, которые работают в R, привыкли к удобным экосистемам вроде **tidymodels** или **mlr3**.

Проблема «Python-зависимости»

Чтобы запустить глубокое обучение в R, приходилось использовать пакет **reticulate** как мост к Python (для вызова TensorFlow или PyTorch).

Что меняет ggmlR

Vulkan — открытый стандарт, который поддерживает практически любое современное **GPU**. Можно запускать модели ONNX.

ggmlR даёт **R-сообществу** нативную, не зависящую от вендора возможность обучать нейросети прямо в **R** и оставаться в привычной среде.

ggmlR позволяет запускать LLM и нейросети на GPU через Vulkan без **reticulate**, без **TensorFlow**, без **PyTorch**, без **CUDA** и без **ROCm**.

Планы

- 1) Обновить апстрим версию ggml → ggmlR
- 2) Сделать пакет для Reinforcement learning
- 3) Обновить зависимые пакеты llamaR и sd2R
- 4) Расширить поддержку ONNX

Спасибо за внимание



GitHub - <https://github.com/Zabis13/ggmlR>

CRAN - <https://cran.r-project.org/web/packages/ggmlR/index.html>

Телеграм: @Zabis13

E-mail: lbsbmsu@mail.ru

<https://t.me/sberlogabig>